My part: slides 7, 8, 9 (Objectives, Architecture, Technologies)

Personal prep sheet. Target length: 2 minutes. The script is about 280 words, which is 2:00 at a calm pace of 140 words per minute. Rehearse with a phone timer. If you run over, drop the lines marked [CUT IF OVER]. Read one line per breath. Bold words get slight stress.

Teleprompter script

[0:00] Slide 7: Objectives

Thank you, Tejas. So those are the gaps. Now, what exactly do we plan to build? We have **five** objectives.

One. Build a role-based web application that manages the **complete** ticket lifecycle.

Two. Fine-tune our **own** small language model to tag each ticket's category and priority automatically.

Three. Route every ticket automatically, using the model's prediction **plus** the agent's current workload.

Four. Enforce SLA policies with **live** timers and escalation alerts.

Five. Capture every agent correction, so the model **retrains** and improves over time.

[CUT IF OVER] Notice that each objective is measurable. [CUT IF OVER] We can test every one of them.

[0:35] Slide 8: Proposed system architecture

[NEXT SLIDE]

Here is how it all fits together. (point left) Customers and agents use a **React** frontend, the dashboards and forms.

The frontend talks to a **FastAPI** server over REST, and WebSockets for live updates.

Inside the server, two parts: the core API gateway, and the **SLA and routing engine** that watches deadlines.

All data lives in **PostgreSQL**: tickets, audit logs, and feedback.

Now the important part. (point right) When a ticket arrives, the backend sends its text to our **in-house** model endpoint.

The model classifies it, drafts a reply, and returns plain **JSON**.

No external API. The data **never** leaves our system.

And there is a loop. Agent corrections, stored in the database, feed a **LoRA** retraining pipeline, which updates the model's weights.

So the system keeps getting **smarter**.

[1:25] Slide 9: Technologies

[NEXT SLIDE]

Quickly, the stack. Frontend: **React** with TypeScript, Material UI or Tailwind, and Socket.io for real-time updates.

Backend: **FastAPI** on Uvicorn, with SQLAlchemy as the ORM. Database: **PostgreSQL**.

For AI: **PyTorch** and Hugging Face. A small model, DistilBERT or Qwen 2.5, fine-tuned with **LoRA**, plus scikit-learn for our **baseline**.

[CUT IF OVER] Everything here is open source, [CUT IF OVER] so the running cost stays near zero.

[1:55] Handover

And with that, Vidhi will take you through the results we expect. Over to you, Vidhi.

Cross-question prep

Answer in two or three sentences, then stop. If a question belongs to results (targets, metrics), give the number and add "Vidhi covers the full evaluation next."

On the objectives (slide 7)

How will you measure these objectives? Each one has a number attached. Classification: macro-F1 of 0.85 or higher. Routing: 80% or more correct first-time assignment. SLA compliance: 15 to 20% improvement. Latency: under 5 seconds per ticket on CPU. The feedback loop is measured as accuracy gain per retraining cycle.

What does role-based mean? Three roles: customer, agent, and admin. Each sees only their own functions. A customer raises and tracks tickets, an agent works the queue, an admin sets SLA policies and views reports.

What is the difference between category and priority? Category says what the ticket is about, which decides the team. Priority says how urgent it is, which decides the SLA clock and the order in the queue. The model predicts both.

On the architecture (slide 8)

Why FastAPI and not Django or Node? It is async Python, so it handles many parallel requests well, and our whole AI stack is Python, so the model plugs in directly. It also generates API docs automatically.

Why both REST and WebSockets? REST for normal request and response, like creating or updating a ticket. WebSockets so the server can push dashboard and SLA updates to agents instantly, without the browser polling.

What exactly does the model endpoint return? A JSON object with the predicted category, the predicted priority, a routing suggestion, and a draft reply. The backend and the routing engine act on that.

What if the model is down or unsure? The ticket still flows. It falls back to the manual queue and our classical baseline, TF-IDF with Logistic Regression, and an agent approves everything anyway. The AI assists; it never blocks the pipeline.

What is LoRA and why not full fine-tuning? LoRA trains small adapter matrices instead of all the model weights, so it needs a fraction of the memory and time. That is what makes periodic retraining affordable, even on Colab.

Walk me through the retraining loop. Agent corrections are saved to a feedback table. A scheduled job exports them as training data, runs a LoRA fine-tune, checks the new model on a held-out set, and if it is better, swaps the weights on the inference endpoint.

Where is security in this diagram? Inside the API gateway: JWT login, role-based access control on every endpoint, and an audit log table that records every action on a ticket.

Why PostgreSQL and not MongoDB? Ticket data is naturally relational: users link to tickets, tickets link to interactions, logs, and SLA policies. We need joins and strict consistency for reports and audits, so PostgreSQL is our primary choice. We listed MongoDB as a considered alternative.

Where does the model run? On the same on-premise deployment, as its own service in Docker. Inference is on CPU, under 5 seconds per ticket, with no external call.

On the technologies (slide 9)

Why TypeScript? Type checking catches bugs at build time, which matters in a dashboard with lots of live state.

What is Uvicorn? The ASGI server that actually runs FastAPI's async code in production.

Why scikit-learn if you use transformers? Two reasons. It builds our baseline, TF-IDF with Logistic Regression, so we can prove the SLM adds real value. And it gives us the evaluation metrics.

DistilBERT or Qwen 2.5, which one will you use? We will benchmark both on the partner's tickets and pick by macro-F1 and latency. DistilBERT is lighter and great for pure classification; Qwen can also generate the draft replies.

Why not simply use the ChatGPT API? Three reasons: per-call cost, latency, and privacy. Confidential ticket text would leave the organization. Keeping the model in-house is the

core idea of this project.

If you get stuck

- Honest fallback: "Good question. That is a design decision we will settle with benchmarks in the early sprints."
- Anchor line: in-house model, data stays private, near-zero cost per ticket, improves from agent feedback.
- Do not argue. Note the point, say you will verify, move on.

Delivery notes

- On slide 8, trace the diagram with your hand: left to right for the main flow, then circle the retraining loop. It fills time naturally and looks confident.
- Pause one full second after "The data never leaves our system." That is your strongest line.
- Before the presentation, fix the slide 9 typo: it says PyTourch, it should say PyTorch. That slide is yours.